🔗 REST & SOAP API Fundamentals

Week 5 • Day 3 • Technical Infrastructure & Modeling

TPM Academy

🎯 Day 3 Objectives

- Use REST **resource modeling** — nouns, not verbs
- Apply HTTP **methods + status codes** consistently
- Design **idempotency** for safe retries
- Pick a **versioning strategy**
- Standardize **error semantics** with the 7 most-needed codes
- Know **when SOAP** is still the right call

🕑 Today's Shape

Clock	Block	Outcome
09:00 – 09:15	Opening	Pin TMD Sections 1–2; recap REST principles
09:15 – 10:30	Block 1 + Activity 1	Resource modeling calibration
10:45 – 12:00	Block 2 + Activity 2	Design your feature's API
13:00 – 14:15	Block 3 + Activity 3	Idempotency, versioning, errors
14:30 – 16:00	Block 4 + Activity 4 + Wrap	REST vs SOAP + AI critique

Block 1 — REST & Resource Modeling

Nouns, not verbs

📚 REST Method Map

Method	Purpose	Idempotent?
GET	Retrieve	Yes
POST	Create	No (typically)
PUT	Replace	Yes
PATCH	Partial update	No (typically)
DELETE	Remove	Yes

URLs are nouns (resources). The verb is the HTTP method. `POST /share` is wrong; `POST /shares` is right.

📊 Status Codes That Carry Meaning

Code	Meaning	When
200	OK	Successful retrieval / update
201	Created	Successful creation
204	No content	Successful with no body (e.g., DELETE)
400	Bad request	Validation failed
401	Unauthenticated	Missing / invalid token
403	Unauthorized	Token valid; lacks permission
404	Not found	Resource doesn't exist
409	Conflict	Idempotency / state collision
422	Unprocessable	Validated but business rule violated
429	Rate limited	Too many requests
500/502/503	Server side	Different shapes of "us, not you"

👥 Activity 1 — Resource Calibration

Quads • 45 minutes

Design the resource model for a generic todo-list API (lists / items / collaborators).

- 20 min: sketch resources + URLs
- 10 min: methods per resource
- 10 min: trickiest design choice (sharing, reordering)
- 5 min: argue + readout prep

⏱️ 20 + 10 + 10 + 5

Block 2 — Your Feature's API

From entities to endpoints

📝 Worked Endpoint (FieldPulse)

```
POST /v1/dispatchers/{dispatcher_id}/reconcile-events

Headers:
Authorization: Bearer <SSO JWT, scope: reconcile.write>
Idempotency-Key: <client UUID>

Body:
ticket_ids: UUID[] (1..100)
submitted_at: ISO 8601 timestamp
client_metadata: { user_agent, location }

Responses:
201 Created — { id, ticket_ids, submitted_at }
400 Validation error — { error.code, fields[] }
401 Missing/invalid token
403 Token lacks reconcile.write scope
409 Idempotency-Key conflict
422 Tickets ineligible
429 Rate limited — Retry-After: <seconds>
```

The Authorization header carries a single sign-on (SSO) token.

👥 Activity 2 — Design Your API

Quads • 50 minutes

- 20 min: list resources from data model
- 10 min: design URL paths
- 10 min: methods + status codes per path
- 10 min: document one endpoint in detail

⏱️ 20 + 10 + 10 + 10

Block 3 — Idempotency, Versioning, Errors

The aspects most often hand-waved

🔄 Idempotency — Safe to Retry

Natural

PUT and DELETE are idempotent by HTTP definition.

Idempotency-Key header

Client sends a UUID; server stores key + response; returns same response on retry.

Conditional

If-Match / If-None-Match gate the operation by version.

For mutating endpoints: decide approach + window (typical: 24 hours).

📚 Versioning Strategies

Strategy	Where version lives	Pros	Cons
URL path	/v1/...	Easy to read; clear breaking	Code duplication
Header	X-API-Version: 2	URL stays clean	Less discoverable
Content negotiation	Accept: ...vnd...;v=2	Standard	Often poorly understood

Default for B2B APIs: URL path versioning.

⚠️ Error Body Shape

```
{
"error": {
"code": "ticket_not_eligible",
"message": "Ticket cannot be reconciled in current state",
"fields": [
{"name": "ticket_ids[3]", "reason": "ticket already reconciled"}
]
},
"request_id": "req_01H...",
"documentation_url": "https://docs.../errors/ticket_not_eligible"
}
```

- **code** — programmatic; clients switch on this
- **message** — human-readable
- **fields** — when validation failed
- **request_id** — for support tickets

👥 Activity 3 — Idempotency, Versioning, Errors

Quads • 50 minutes

- 25 min: idempotency per mutating endpoint + window
- 10 min: pick versioning strategy
- 15 min: error body shape + 7 most-needed codes

⏱️ 25 + 10 + 15

Block 4 — REST vs SOAP + AI Critique

When SOAP is still the right answer

📚 When SOAP Is Still the Right Call

- Integration partner only speaks SOAP (banks, government, Electronic Data Interchange (EDI)-adjacent)
- WS-Security or WS-ReliableMessaging contractually required
- Partner has invested heavily in SOAP tooling; rewrite is impractical

> **For most new B2B features:** REST + JSON. SOAP appears occasionally for legacy enterprise integrations. Document the choice honestly.

🤖 AI Critique Prompt

```
Role: API design reviewer with strong REST opinions.
Context: <paste URL paths, methods, status codes, error body
shape, idempotency approach, versioning strategy>
Task: Identify the top 5 issues in this API contract.
Constraints:
- Be specific: cite the path, method, or convention
- For each, suggest the correction
- Avoid pedantic preferences (singular/plural is fine
if consistent)
Format: Numbered — Issue / What's wrong / Fix.
```

👥 Activity 4 — SOAP + AI Critique

Quads • 55 minutes • Wrap

- 25 min: SOAP question — any partner needs it?
- 10 min: AI critique → 5 issues
- 10 min: adopt / defer / reject
- 10 min: polish Section 3 + provenance

⏱️ 25 + 10 + 10 + 10 · 15-min wrap

🎉 Day 3 Wrap

What we built

- Resource design with URLs
- Methods + status codes
- One detailed endpoint
- Idempotency / versioning / error semantics
- SOAP exceptions documented
- Technical Modeling Document (TMD) Section 3 drafted

What opens tomorrow

- **High-level technical modeling**
- Sequence diagrams — happy + sad/weird
- Putting data + cloud + APIs together

Bring to Day 4: TMD Sections 1–3. Tomorrow we draw the sequences end-to-end.

💬 Questions & Reflection

Which of your endpoints would a hostile integration partner break first?